HM Revenue
& Customs

jbi training

Introduction to Git 2 Days

Your course is being prepared …

Your instructor is : **Peter**

Cameras on please

Timings:

Start 09:30
Lunch 12:30 -13:30
Finish 16:00 -16:30

Outline

I. Introduction to source control

A. History and fundamental concepts behind source control

B. Centralized vs. distributed version control

II. Introduction to Git

A. What is Git? Basic Git concepts and architecture

B. Git workflows: Creating a new repo (adding, committing code)

C. HEAD

D. Git commands (checking out code)

E. Master vs branch concept

F. Creating a branch/switching between branches

G. Merging branches and resolving conflicts

III. Introduction to GitHub

A. What is GitHub? Basic GitHub concepts

B. GitHub in practice: Distributed version control

C. Cloning a remote repo

D. Fetching/Pushing to a remote repo

E. Collaborating using Git and GitHub

What is a ‘version control system?’

- a way to manage files and directories
- track changes over time
- recall previous versions
- ‘source control’ is a subset of a VCS.

Some history of source control…

(1972) Source Code Control System (SCCS)
- closed source, part of UNIX

(1982) Revision Control System(RCS)
- open source

(1986) Concurrent Versions System (CVS)
- open source

(2000) Apache Subversion (SVN)
- open source

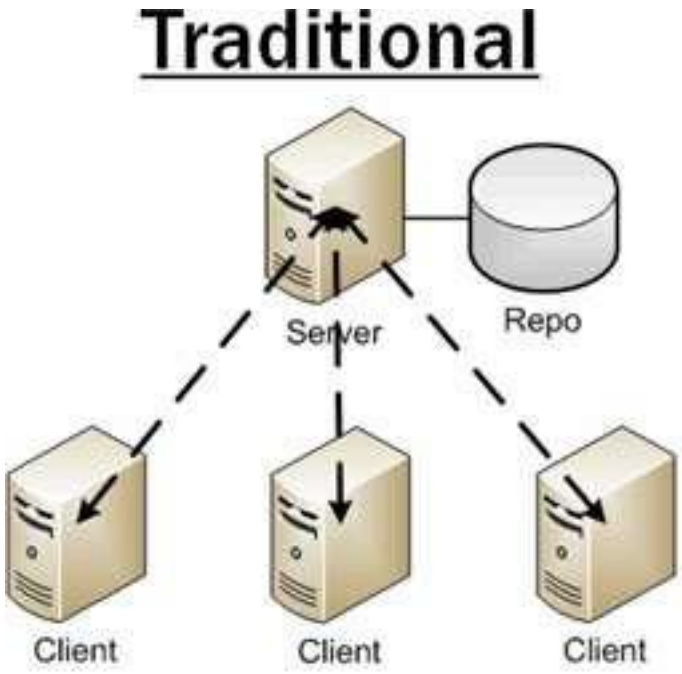
…more history

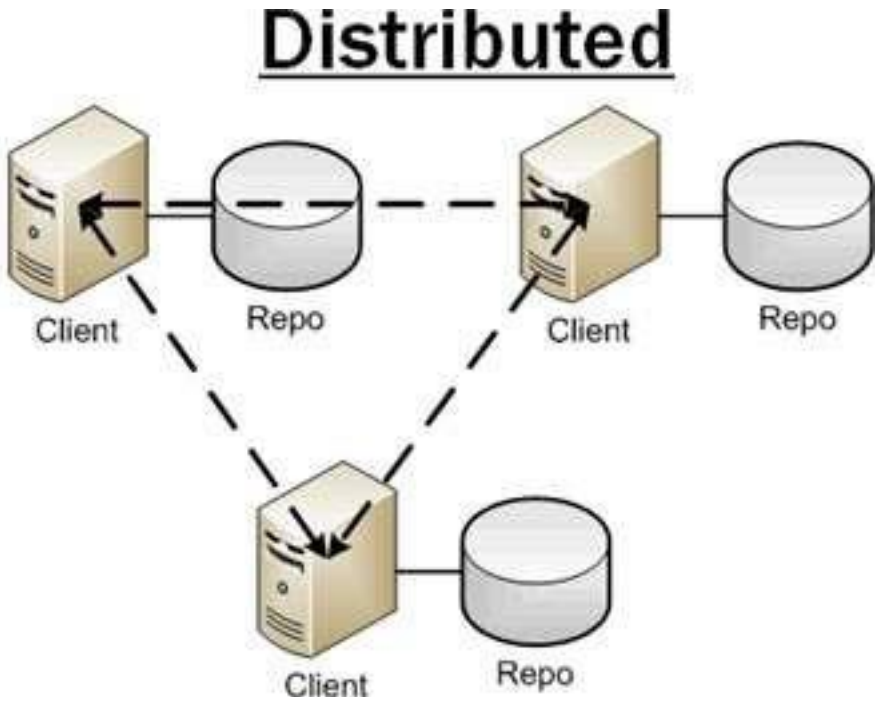
(2000) BitKeeper SCM

- closed source, proprietary, used with source code management of Linux kernel
- free until 2005
- distributed version control

Distributed version control

No central server
Every developer is a client, the server and the repository





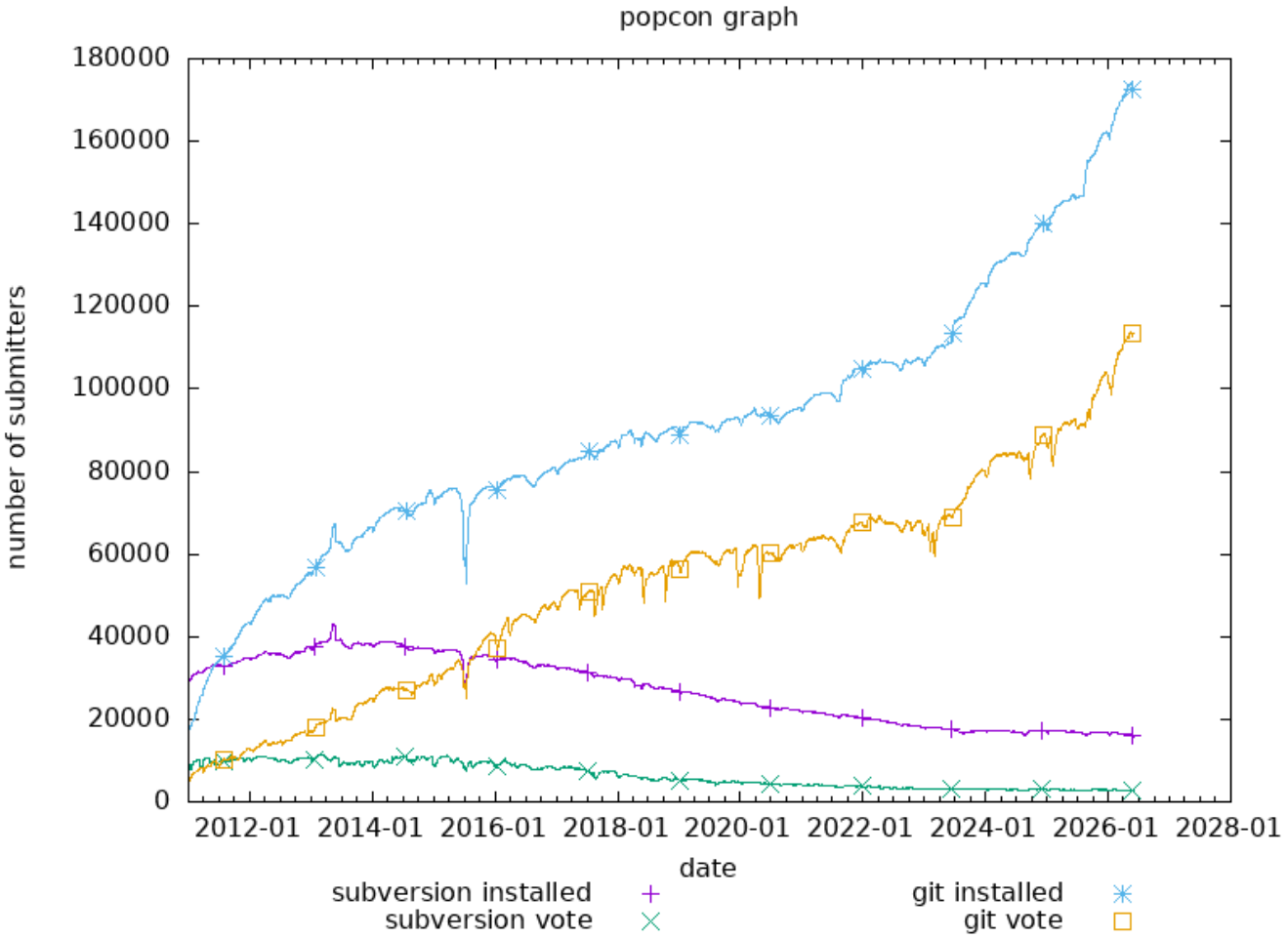
Source: http://bit.ly/1SH4E23

What is git?

What is git?

- created by Linus Torvalds, April 2005
- replacement for BitKeeper to manage Linux kernel changes
- a command line version control program
- uses checksums to ensure data integrity
- distributed version control (like BitKeeper)
- cross-platform
- open source, free

Popularity



https://www.openhub.net/repositories/compare
http://bit.ly/1QyLoOu

Git distributed version control

- "If you're not distributed, you're not worth using." – Linus Torvalds
- no need to connect to central server
- can work without internet connection
- no single failure point
- developers can work independently and merge their work later
- every copy of a Git repository can serve either as the server or as a client (and has complete history!)
- Git tracks **changes**, not versions
- Bunch of little **change sets** floating around

Is Git for me?

- People primarily working with source code

- Anyone wanting to track edits (especially changes to text files)
  - review history of changes
  - anyone wanting to share, merge changes

- Anyone not afraid of
  command line tools

Most popular languages used with Git

- HTML
- CSS
- Javascript
- Python
- ASP
- Scala
- Shell scripts
- PHP
- Ruby
- Ruby on Rails
- Perl
- Java
- C
- C++
- C#
- Objective C
- Haskell
- CoffeeScript
- ActionScript

Not as useful for image, movies, music…and files that must be interpreted (.pdf, .psd, etc.)

How do I get it?

http://git-scm.com

Git install tip

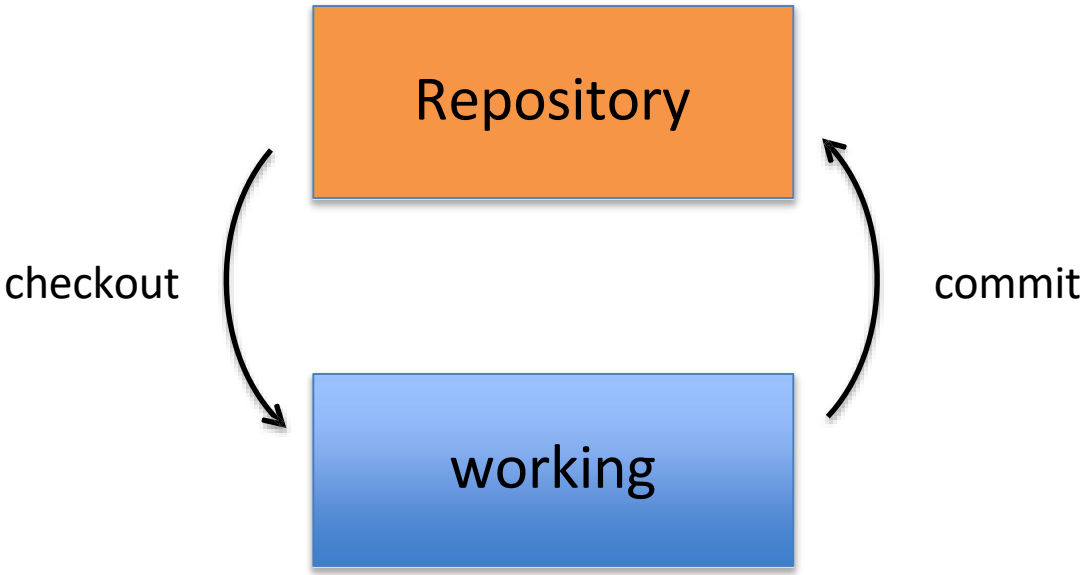
- Much better to set up on a per-user basis (instead of a global, system-wide install)

What is a repository?

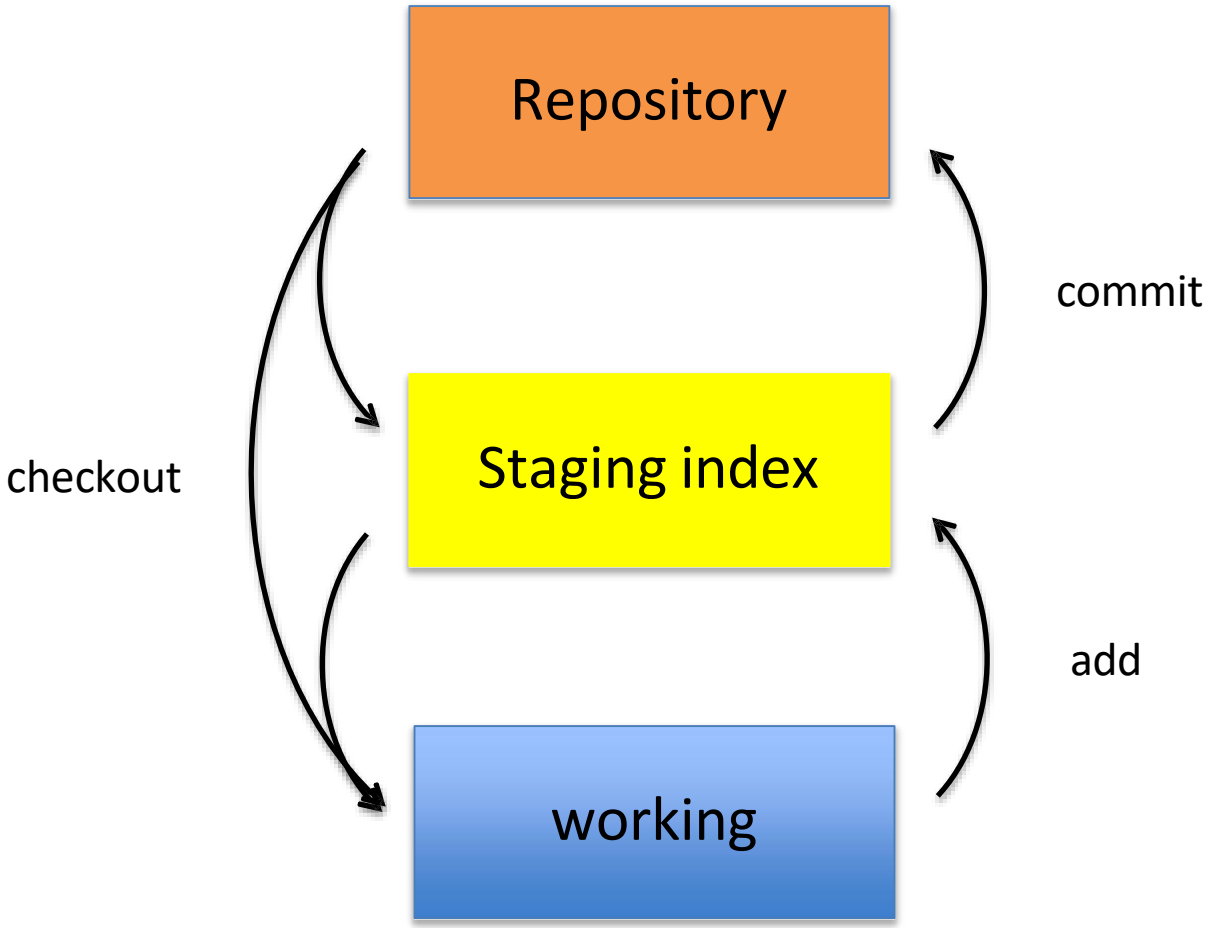
- “repo” = repository
- usually used to organize a single project
- repos can contain folders and files, images, videos, spreadsheets, and data sets – anything your project needs

Two-tree architecture

other VCSs



Git uses a three-tree architecture



A simple Git workflow

1. Initialize a new project in a directory:

   git init

```
peter@Ratty MINGW64 ~
$ mkdir  gitdemo

peter@Ratty MINGW64 ~
$ cd gitdemo/

peter@Ratty MINGW64 ~/gitdemo
$ git init
Initialized empty Git repository in C:/Users/peter/gitdemo/
git/

peter@Ratty MINGW64 ~/gitdemo (master)
```

2. Add a file using a text editor to the directory
3. Add every change that has been made to the directory:

   git add .

4. Commit the change to the repo:

   git commit –m “important message here”

   .

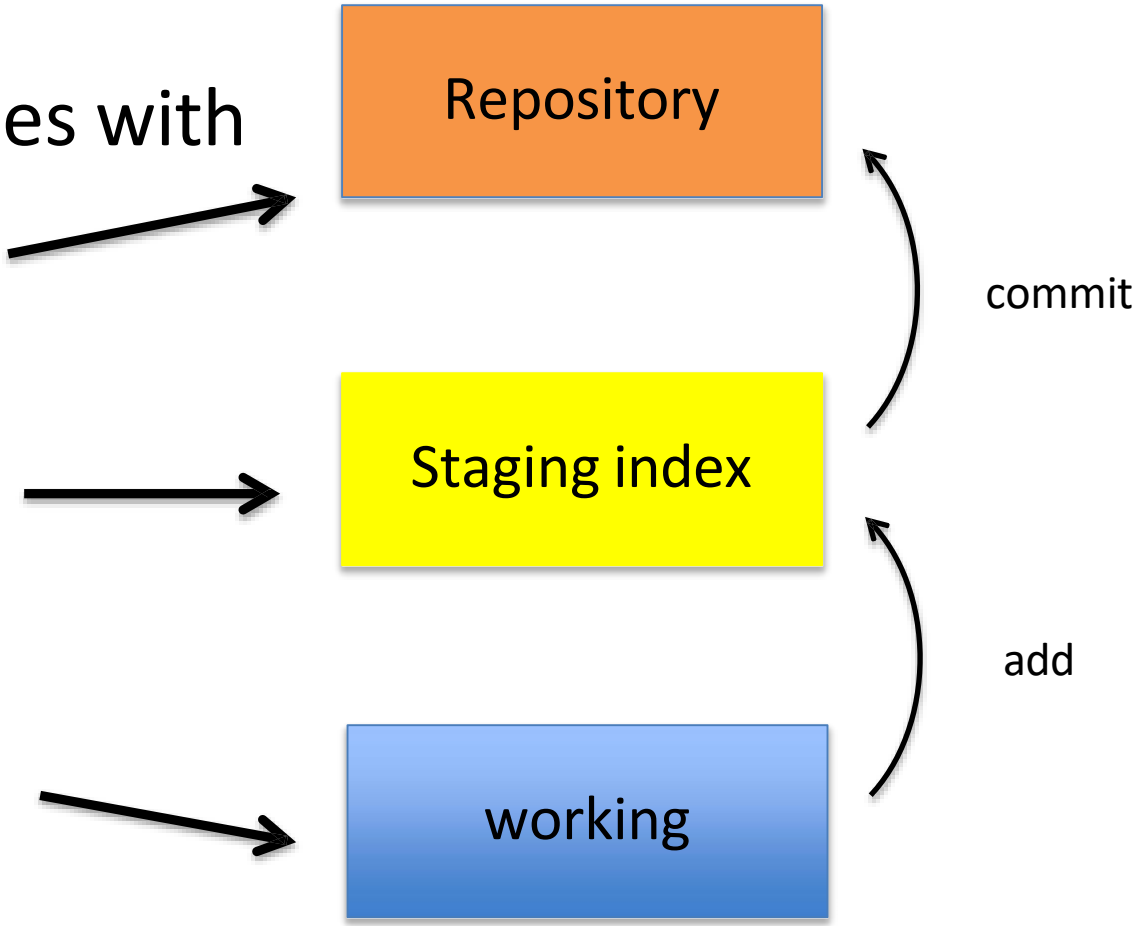
```
$ git add .
warning: in the working copy of 'f1', LF will be replaced by
 CRLF the next time Git touches it

peter@Ratty MINGW64 ~/gitdemo (master)
$ git commit -m 'added first file'
[master (root-commit) ede9efc] added first file
 1 file changed, 1 insertion(+)
 create mode 100644 f1
```

After initializing a new git repo…

3. Commit changes with a message

2. Add changes

1. Make changes



Mini Lab 1

- Create a folder in your the home directory **cd** called **firstRepo**.
- Initialise it as a Git repository.
- Check the status of the repo.
- Use the **touch** command to create a file called **myfile.bat**.
- Check the status of the repo.

Mini Lab 2

- Create a folder in your the home directory **cd** called **firstRepo**.
- Initialise it as a Git repository.
- Check the status of the repo.
- Use the **touch** command to create a file called **myfile.bat**.
- Check the status of the repo.

Mini Lab 3

- Create a new directory called **wipeout** in your Home folder.
- Initialise as a repo.
- Create a file called **f1** and add a line of text to it.
- Stage the file for commit and check the status.
- Commit the repo.
- Create a second empty file **f2** with one line of text and add it to the repo.
- Add an additional line of text to the file.
- Use the diff command and note what it is telling you.
- Stage the file.
- Use the diff command again.
- Commit the changes.

Mini Lab 4

- In your most recent repo, use the one line log to locate the SHA1 history.
- Move the HEAD to the start of the commit history via checkout.
- Check the log history.
- Use touch to create a new file.
- Add it to the repo.
- Commit the changes.
- Use checkout to return to the Master.
- Check the contents of your directory and note what has happened to the new file.
- Check the log history.

Mini Lab 5

- Create a new directory and repo called **stashing**.
- Use vi to create a new file called **mywip** and add a line to it.
- Add the file then commit the branch.
- Modify the file again and commit the branch.
- Use the log to find the original SHA1 key.
- Move the head to that location and create a new branch. (checkout)
- Modify mywip and save.
- Try to switch to the master branch – what happens?
- Stash the current wip – what happens?
- Now switch to the master branch.
- Check the status of the stash.
- Pop the most recent sashed operation.
- Change the file and commit.
- Switch to the Master.

A note about commit messages

- Tell what it does (present tense)
- Single line summary followed by blank space followed by more complete description
- Keep lines to <= 72 characters
- Ticket or bug number helps

Good and bad examples

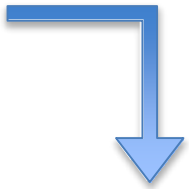
Bad: “Typo fix”

Good: “Add missing / in CSS section”

Bad: “Updates the table. We’ll discuss next Monday with Sam.”

Bad: git commit -m "Fix login bug”

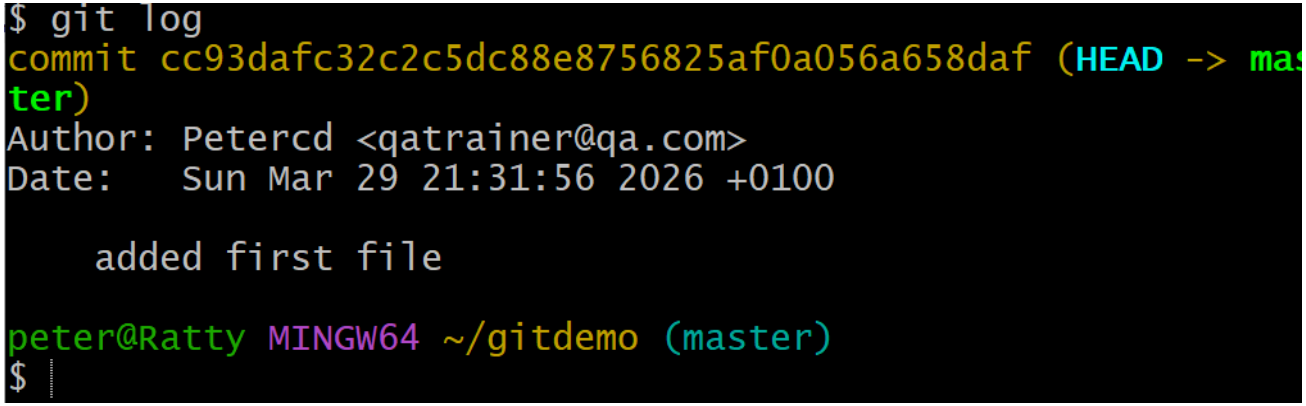
Good: git commit -m



```
Redirect user to the requested page after login

https://trello.com/path/to/relevant/card

Users were being redirected to the home page after login, which is less
useful than redirecting to the page they had originally requested before
being redirected to the login form.

* Store requested path in a session variable
* Redirect to the stored location after successfully logging in the user
```

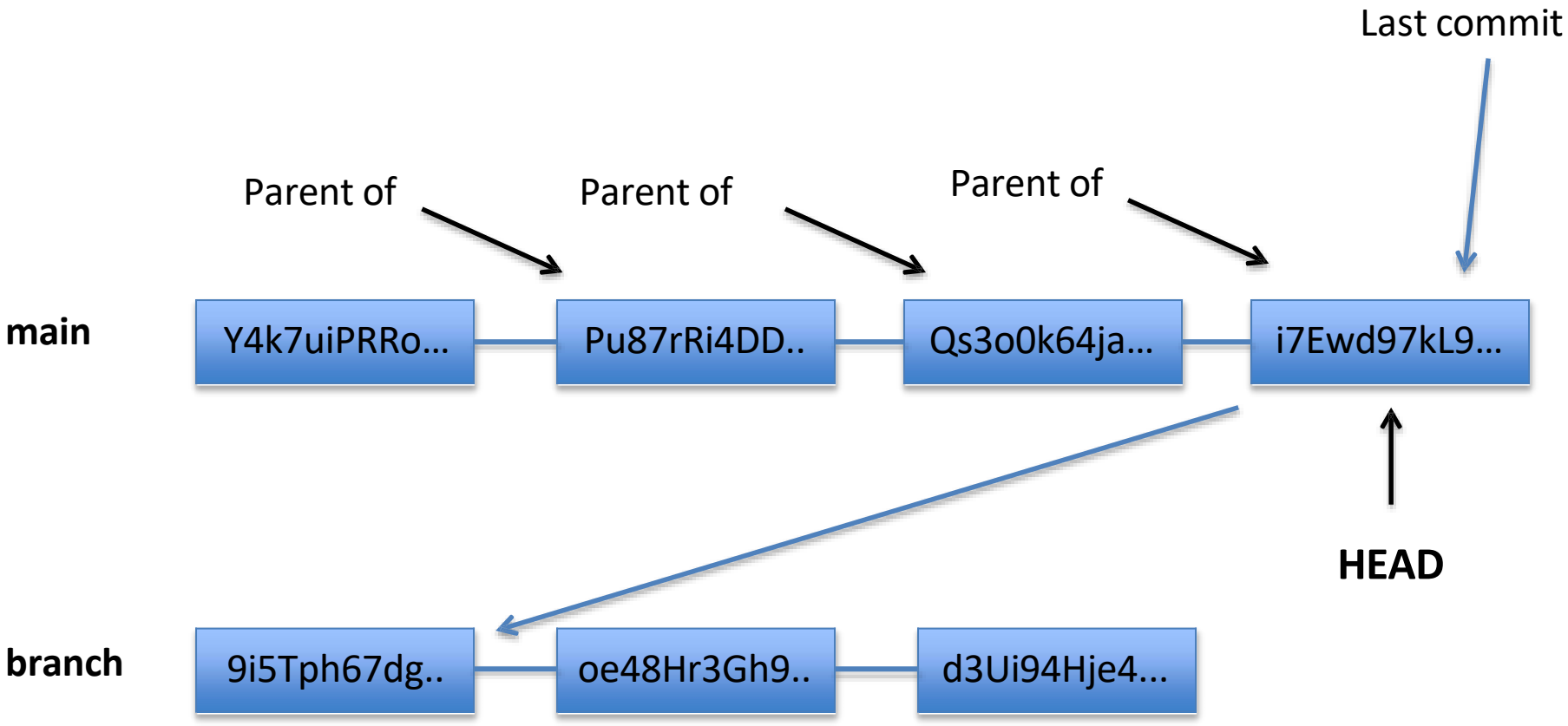
How to see what was done?

git log



The HEAD pointer

- points to a specific commit in repo
- as new commits are made, the pointer changes
- **HEAD always points to the "tip" of the currently checked-out branch in the repo**
- (not the working directory or staging index)
- last state of repo (what was checked out initially)
- HEAD points to parent of next commit (where writing the next commit takes place)

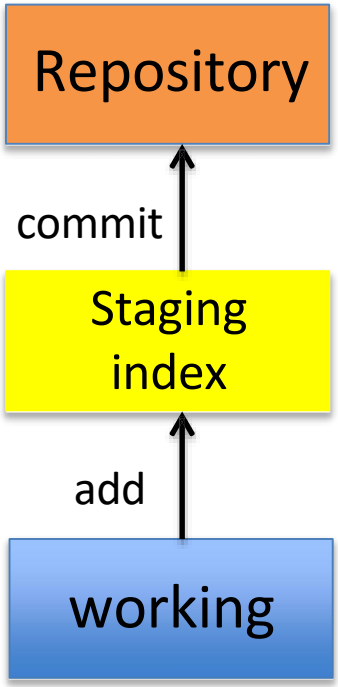

Last commit
Parent of
Parent of
Parent of
main
Y4k7uiPRRo...
Pu87rRi4DD..
Qs3o0k64ja...
i7Ewd97kL9...
HEAD
branch
9i5Tph67dg..
oe48Hr3Gh9..
d3Ui94Hje4...

Which files were changed and where do they sit in the three tree?

git status – allows one to see where files are in the three tree scheme

```
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be commit
ted)
        f3

nothing added to commit but untracked files present (use "gi
t add" to track)

peter@Ratty MINGW64 ~/gitdemo (master)
$
```

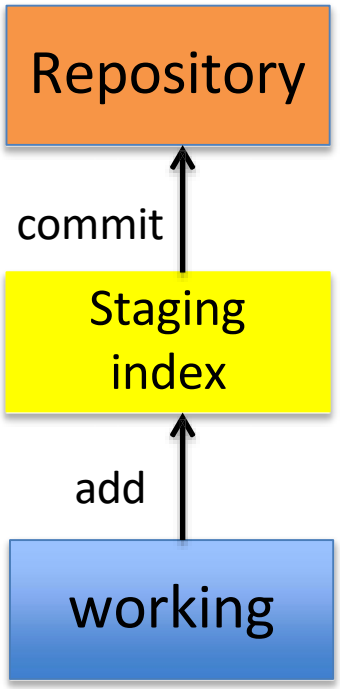
```
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   f3


peter@Ratty MINGW64 ~/gitdemo (master)
$
```



What changed in working directory?

git diff – compares changes to files between repo and working directory

```
$ git diff f3
warning: in the working copy of 'f3', LF will be replaced by CRLF the next
 time Git touches it
diff --git a/f3 b/f3
index 3027a6e..e5ac6d0 100644
--- a/f3
+++ b/f3
@@ -1,3 +1,5 @@
 titest

 Some Text
+
+more text
```



Note: git diff --staged - compares staging index to repo

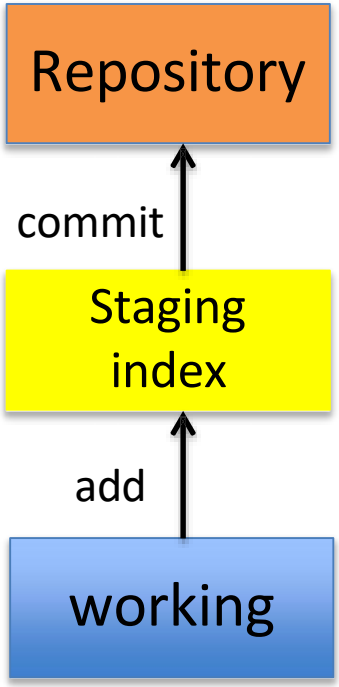
Note: git diff filename can be used as well

Deleting files from the repo

git rm filename.txt

- moves deleted file change to staging area

- It is not enough to delete the file in your working directory. You must commit the change.



Deleting files from the repo

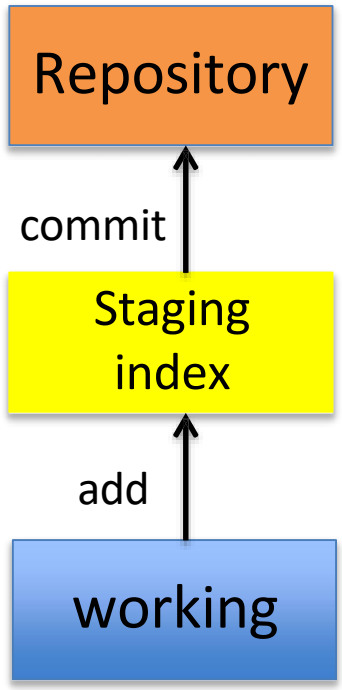
```
$ git rm f3
rm 'f3'

peter@Ratty MINGW64 ~/gitdemo (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    f3
```

Moving (renaming) files

git mv filename1.txt filename2.txt

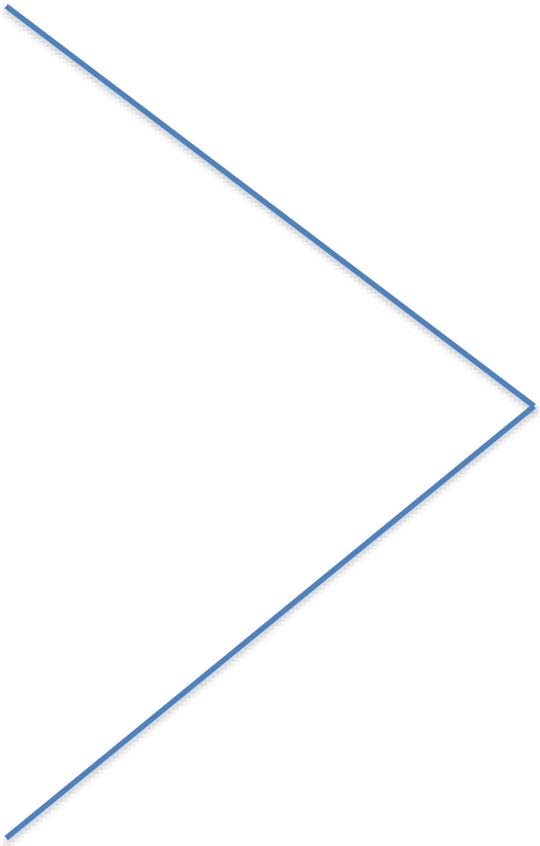
```
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    f3
        renamed:    f2 -> f4


peter@Ratty MINGW64 ~/gitdemo (master)
```

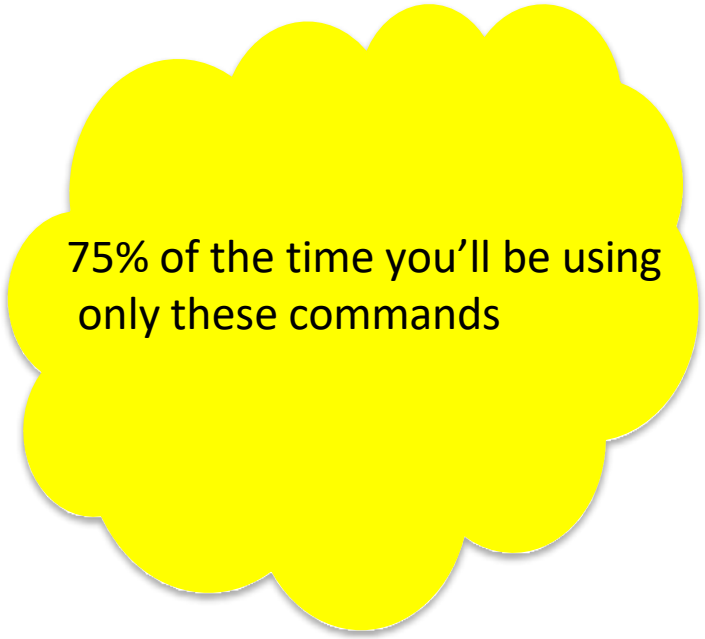
Note: File f3 was committed to repo earlier.



Good news!

git init
git status
git log
git add
git commit
git diff
git rm
git mv





What if I want to undo changes made to working directory?

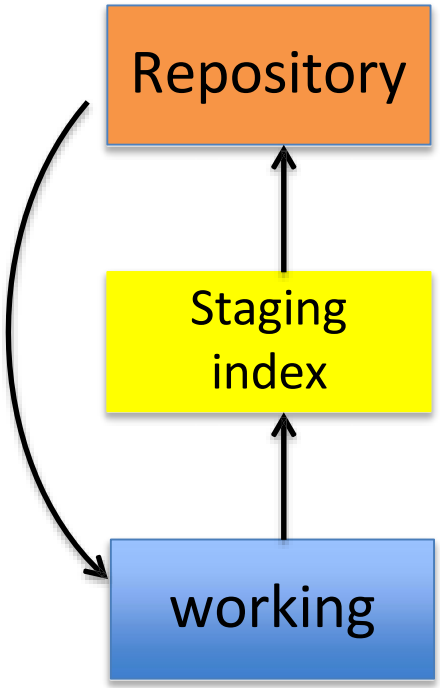
git checkout *something*

(where "something" is a file or an entire branch)

- git checkout will grab the file from the repo

- *Example:* git checkout -- file1.txt

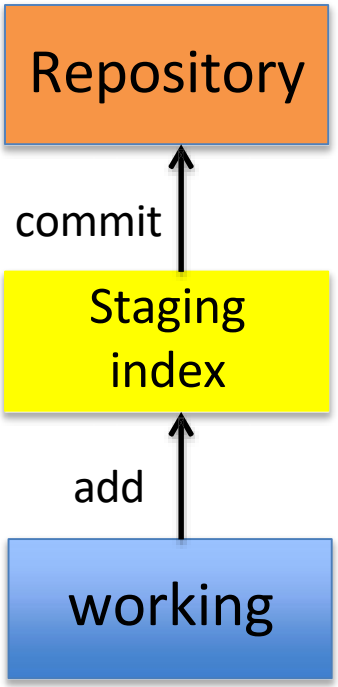
("checkout file 'file1.txt' from the current branch")



What if I want to undo changes added to staging area?

git reset HEAD filename.txt

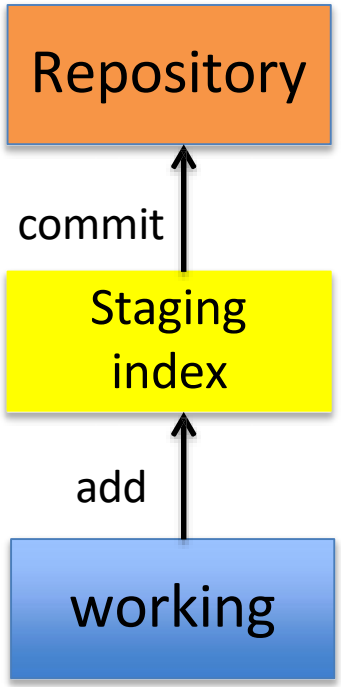
```
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   f4

no changes added to commit (use "git add" and/or "git commit -a")

peter@Ratty MINGW64 ~/gitdemo (master)
$ git reset HEAD f4
Unstaged changes after reset:
M       f4

peter@Ratty MINGW64 ~/gitdemo (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   f4

no changes added to commit (use "git add" and/or "git commit -a")
```



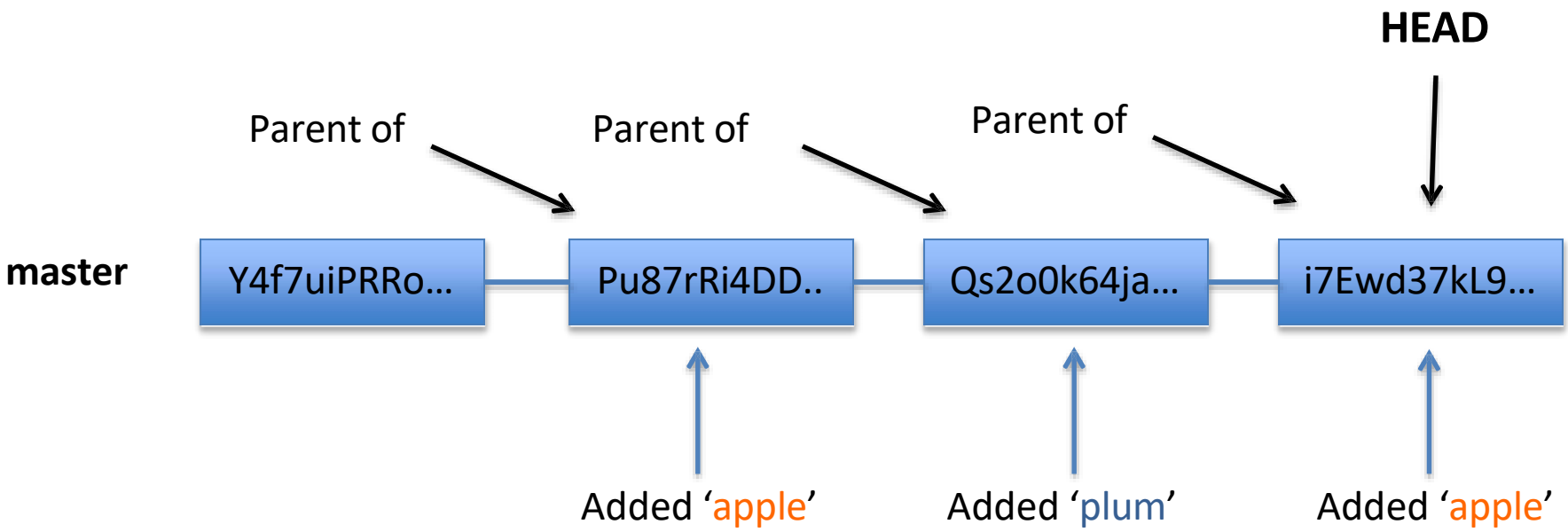
What if I want to undo changes committed to the repo?

git commit --amend -m “message”

- allows one to amend a change to the last commit
- anything in staging area will be amended to the last commit

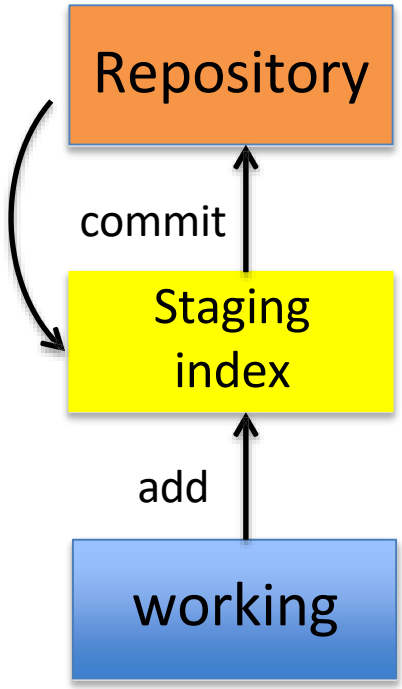


Note: To undo changes to older commits, make a new commit



Obtain older versions

```
no changes added to commit (use "git add" and/or "git commit -a")

peter@Ratty MINGW64 ~/gitdemo (master)
$

peter@Ratty MINGW64 ~/gitdemo (master)
$ git log
commit 69ea5cea86ecb44ef96ce693b369ec20e369d018 (HEAD -> master)
Author: Petercd <qatrainer@qa.com>
Date:   Sun Mar 29 21:54:40 2026 +0100

    added temp files

commit aeb7f79b70ab3a370b0b72f82f15a8258bc791ea
Author: Petercd <qatrainer@qa.com>
Date:   Sun Mar 29 21:52:46 2026 +0100

    new files.

commit dec5cbd416c068e1a02639881c709b8c205f0386
Author: Petercd <qatrainer@qa.com>
Date:   Sun Mar 29 21:46:40 2026 +0100
```

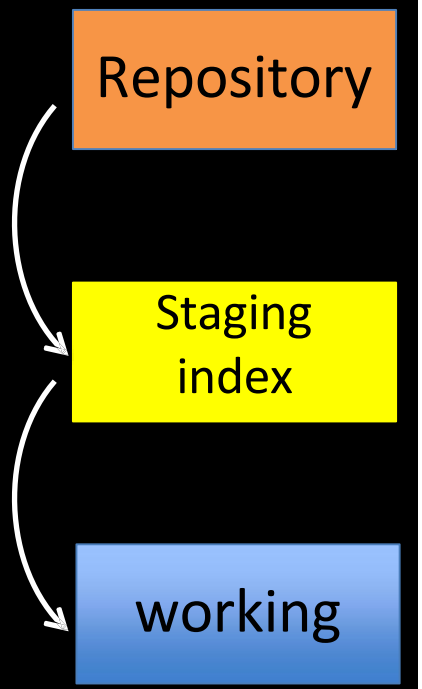


git checkout 856cf235a2-- filename.txt

Note: Checking out older commits places them into the staging area

git checkout 856cf23 -- f4

```
peter@Ratty MINGW64 ~/gitdemo (master)
$ git checkout 856cf23 -- f3

peter@Ratty MINGW64 ~/gitdemo (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   f3

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   f4


peter@Ratty MINGW64 ~/gitdemo (master)
$ git diff
warning: in the working copy of 'f4', LF will be replaced by CRLF the next time Git
touches it
diff --git a/f4 b/f4
index f3d9879..a2e97ab 100644
--- a/f4
+++ b/f4
@@ -1,3 +1,5 @@
 some new text

 more text
+
+and more
```



Which files are in a repo?

git ls-tree tree-ish

tree-ish – a way to reference a repo

full SHA, part SHA, HEAD, others

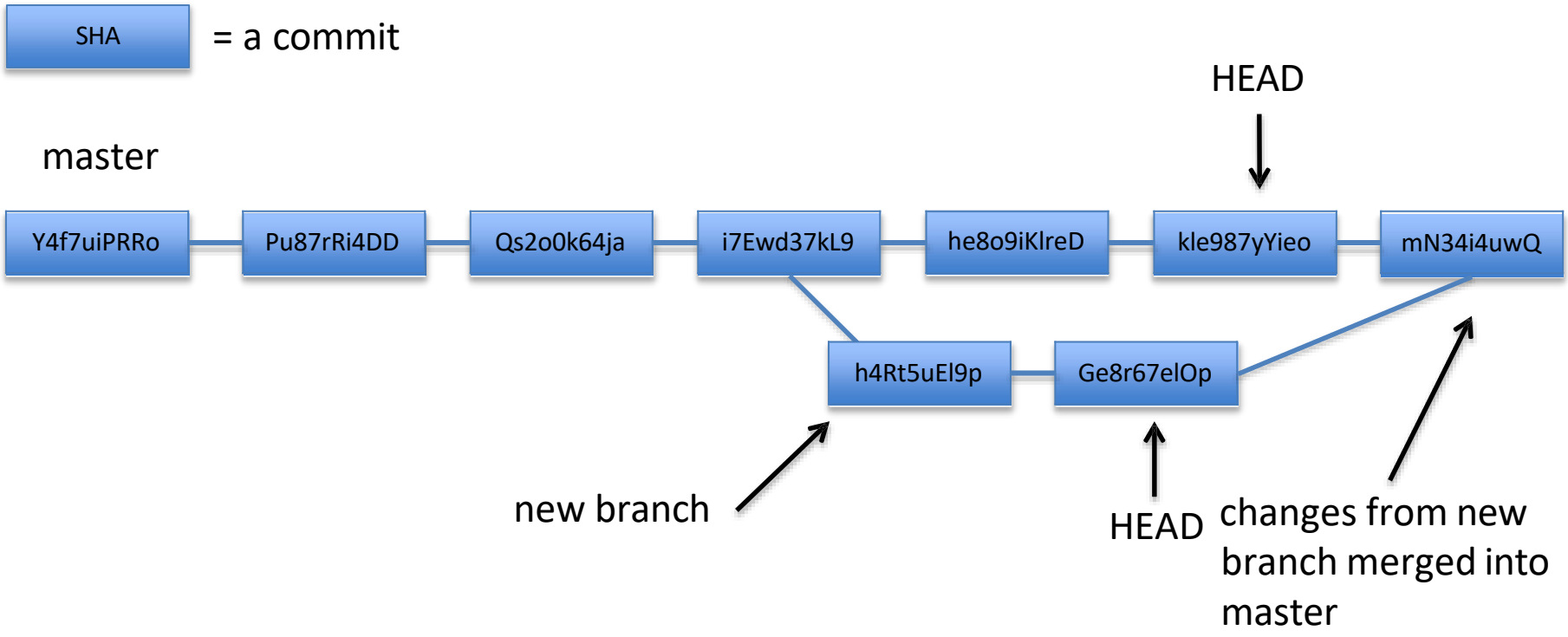
```
peter@Ratty MINGW64 ~/gitdemo (master)
$ git ls-tree HEAD
100644 blob 7d48ced29748e2ab5d429ddcf97a5a3bff9d9a2c    f1
100644 blob f3d9879e55b8e903484366a34b428a0a8a9d8c0f    f4
```

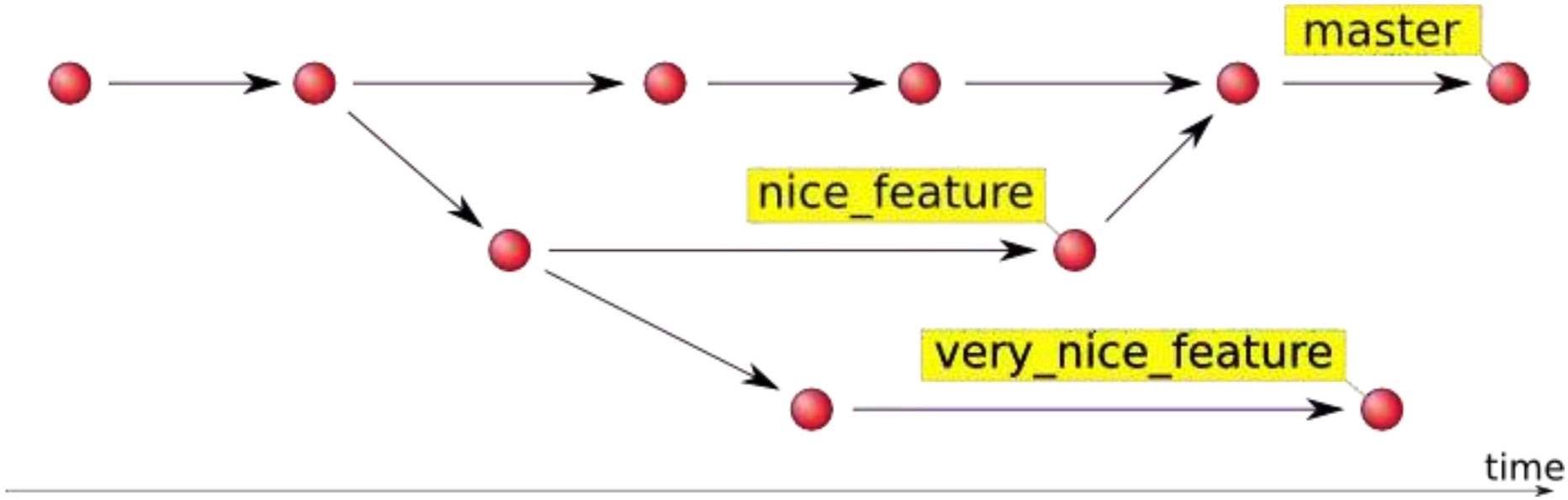
blob = file, tree = directory

branching

- allows one to try new ideas
- If an idea doesn't work, throw away the branch. Don't have to undo many changes to master branch
- If it *does* work, merge ideas into master branch.
- There is only one working directory

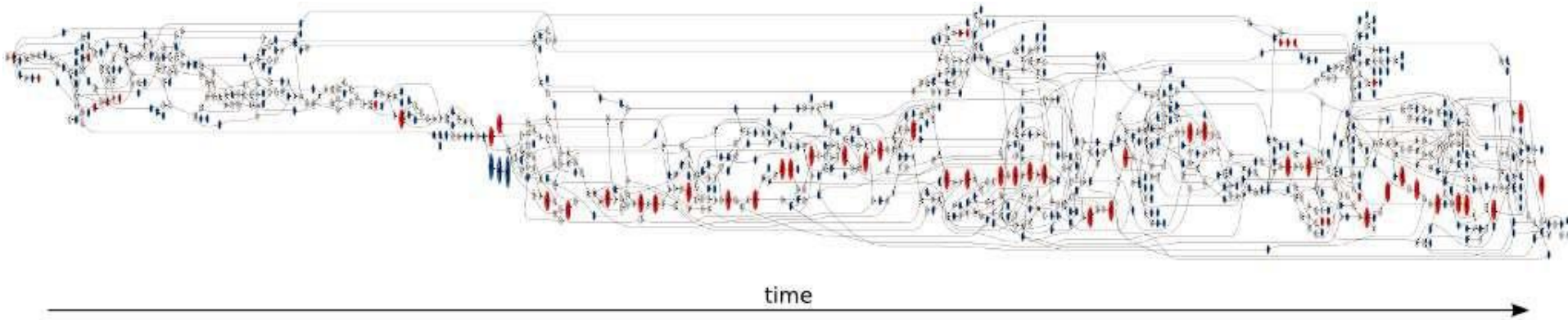
Branching and merging example





Source: https://hades.github.io/2010/01/git-your-friend-not-foe-vol-2-branches

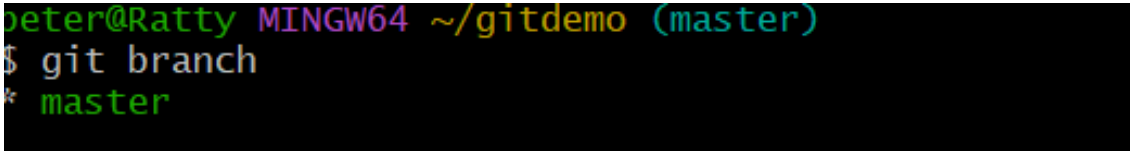
16 forks and 7 contributors to the master branch



red - commits pointed to by tags
blue - branch heads
white - merge and bifurcation commits.

Which branch am I in ?

git branch --list



How do I create a new branch?

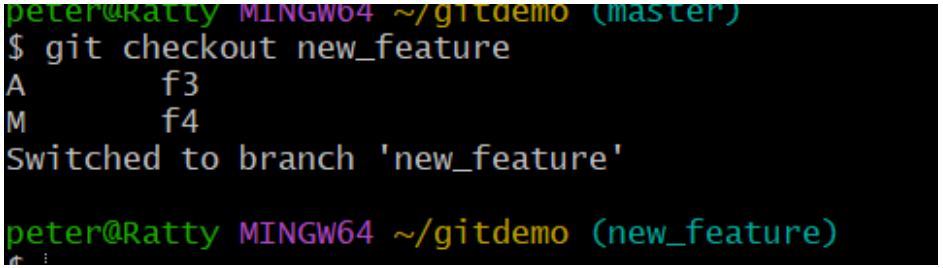
git branch new_branch_name

```
peter@Ratty MINGW64 ~/gitdemo (master)
$ git branch new_feature

peter@Ratty MINGW64 ~/gitdemo (master)
$ git branch
* master
  new_feature
```

Note: At this point, both HEADs of the branches are pointing to the same commit (that of master)

How do I switch to new branch?

git checkout new_branch_name



At this point, one can switch between branches, making commits, etc. in either branch, while the two stay separate from one another.

Note: In order to switch to another branch, your current working directory must be clean (no conflicts, resulting in data loss).

Comparing branches

git diff first_branch..second_branch

```
peter@Ratty MINGW64 ~/gitdemo (new_feature)
$ git diff master new_feature
diff --git a/f3 b/f3
new file mode 100644
index 0000000..63fb837
--- /dev/null
+++ b/f3
@@ -0,0 +1 @@
+titest
diff --git a/f4 b/f4
index f3d9879..a2e97ab 100644
--- a/f4
+++ b/f4
@@ -1,3 +1,5 @@
 some new text

 more text
+
+and more
diff --git a/newcss b/newcss
new file mode 100644
index 0000000..69eb0ad
--- /dev/null
+++ b/newcss
@@ -0,0 +1 @@
```

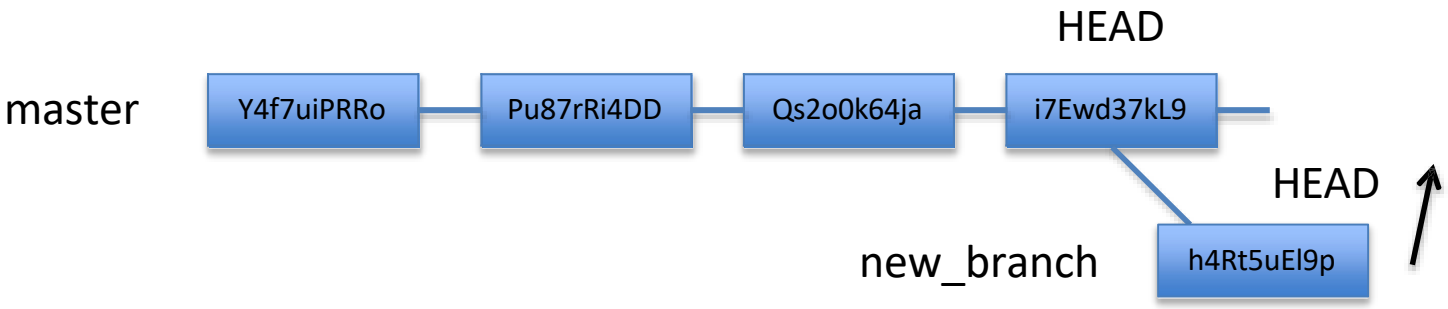
How do I merge a branch?

From the branch into which you want to merge another branch….

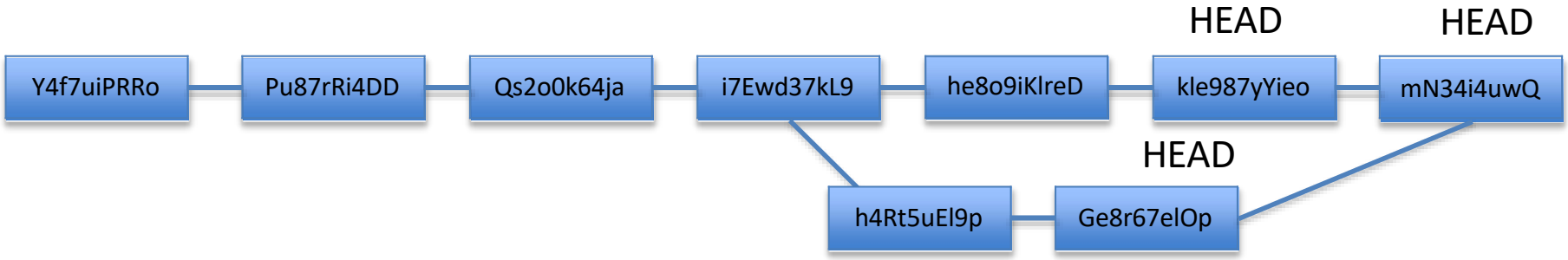
git merge branch_to_merge

Note: Always have a clean working directory when merging

"**fast-forward**" merge occurs when HEAD of master branch is seen when looking back

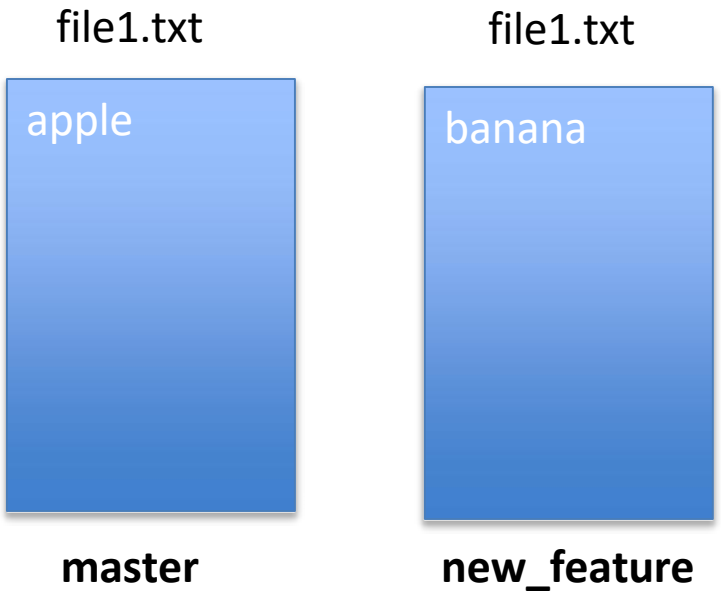


"**recursive**" merge occurs by looking back and combining ancestors to resolve merge



merge conflicts

What if there are two changes to same line in two different commits?




```
peter@Ratty MINGW64 ~/gitdemo (master)
$ git merge new_feature
Updating 38db2a9..c2fe9f2
Fast-forward
 newcss | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)

peter@Ratty MINGW64 ~/gitdemo (master)
```

Resolving merge conflicts

Git will notate the conflict in the files!

```
<<<<<<< HEAD
apple
=======
banana
>>>>>>> new_feature
```

Solutions:

1. Abort the merge using git merge –abort
2. Manually fix the conflict
3. Use a merge tool (there are many out there)

Graphing merge history

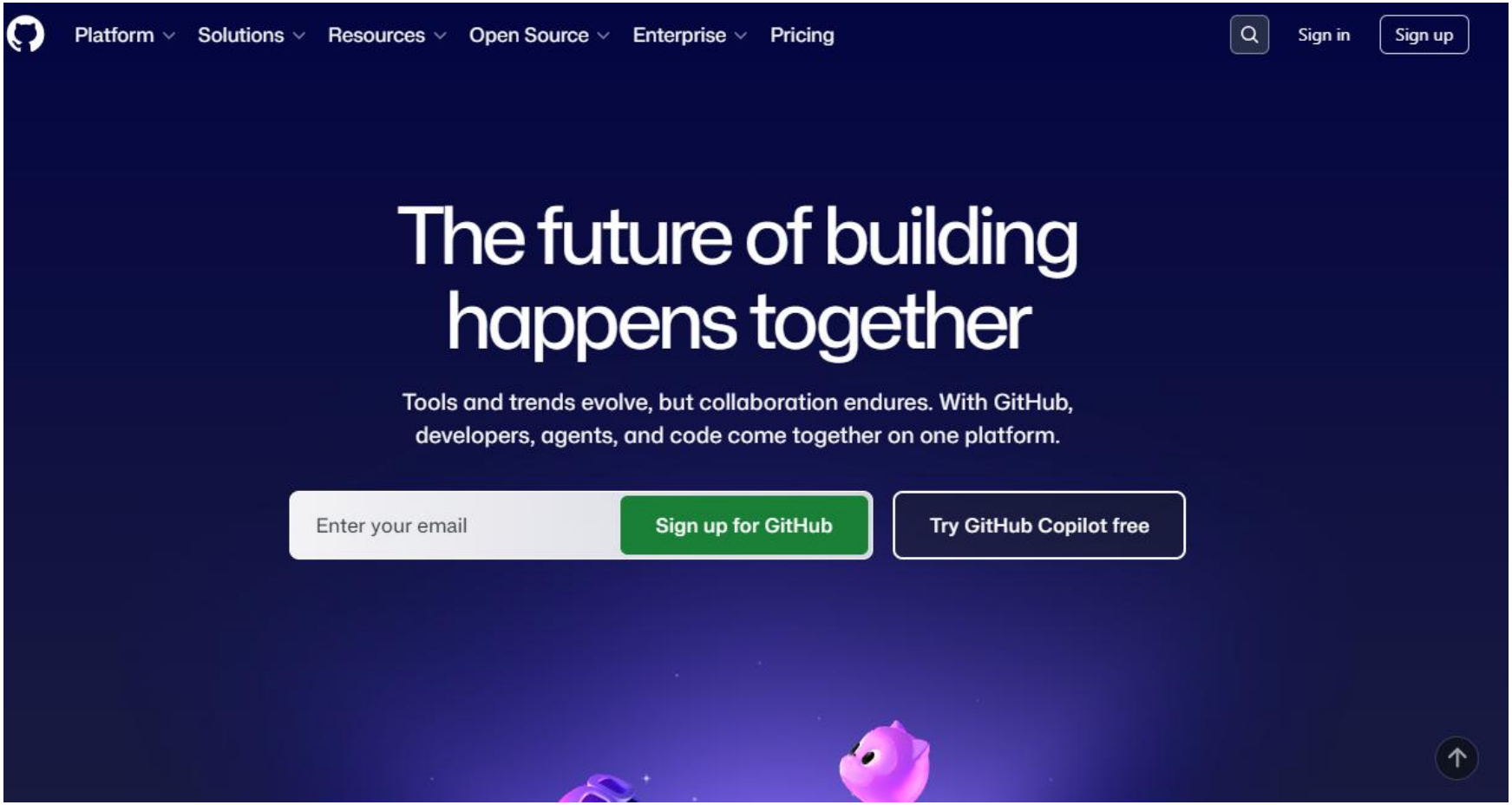
git log --graph --oneline --all --decorate

Tips to reduce merge pain

- merge often
- keep commits small/focused
- bring changes occurring to master into your branch frequently (“tracking”)

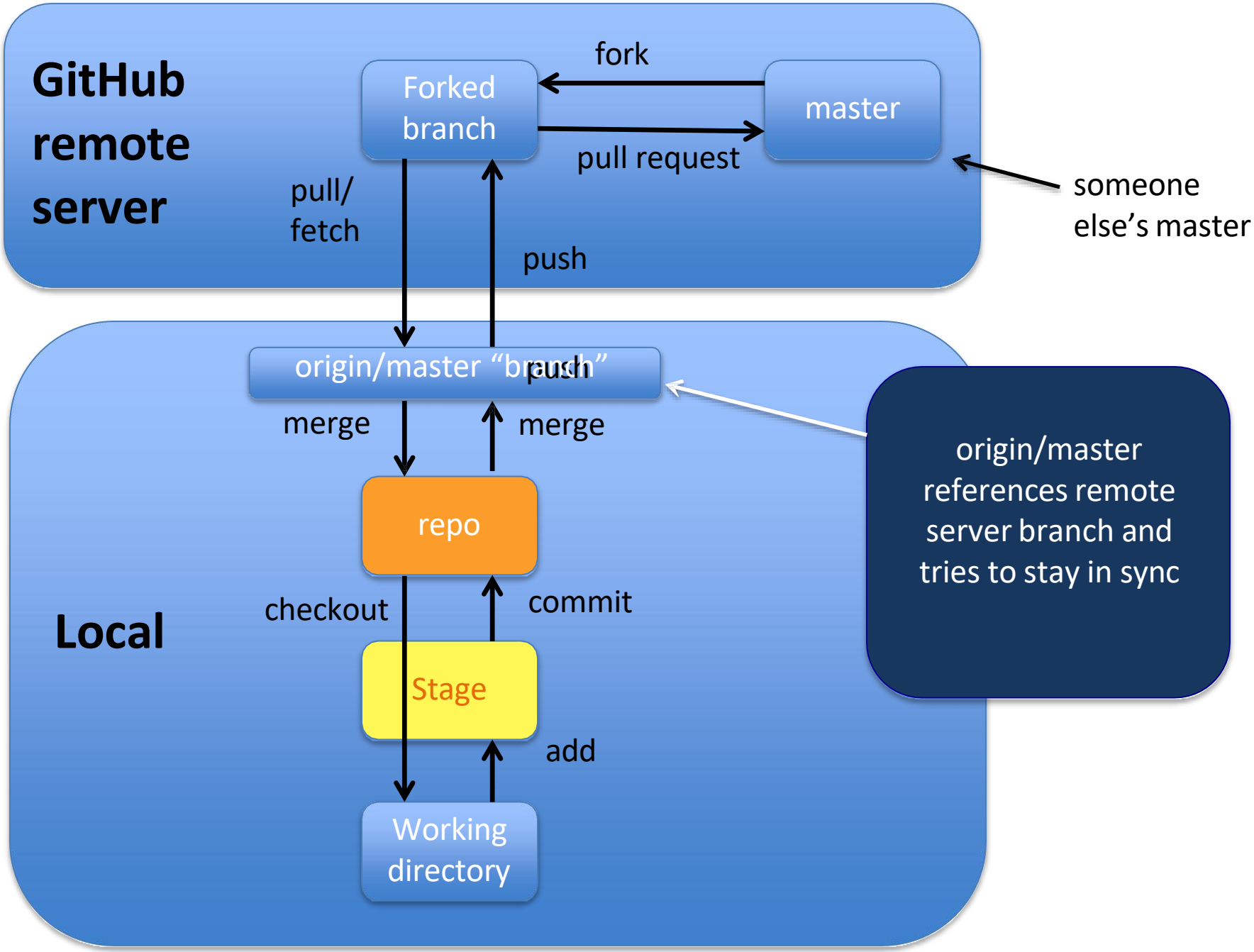
What is  ?

GitHub


Platform
Solutions
Resources
Open Source
Enterprise
Pricing
Sign in
Sign up
The future of building happens together
Tools and trends evolve, but collaboration endures. With GitHub, developers, agents, and code come together on one platform.
Enter your email
Sign up for GitHub
Try GitHub Copilot free

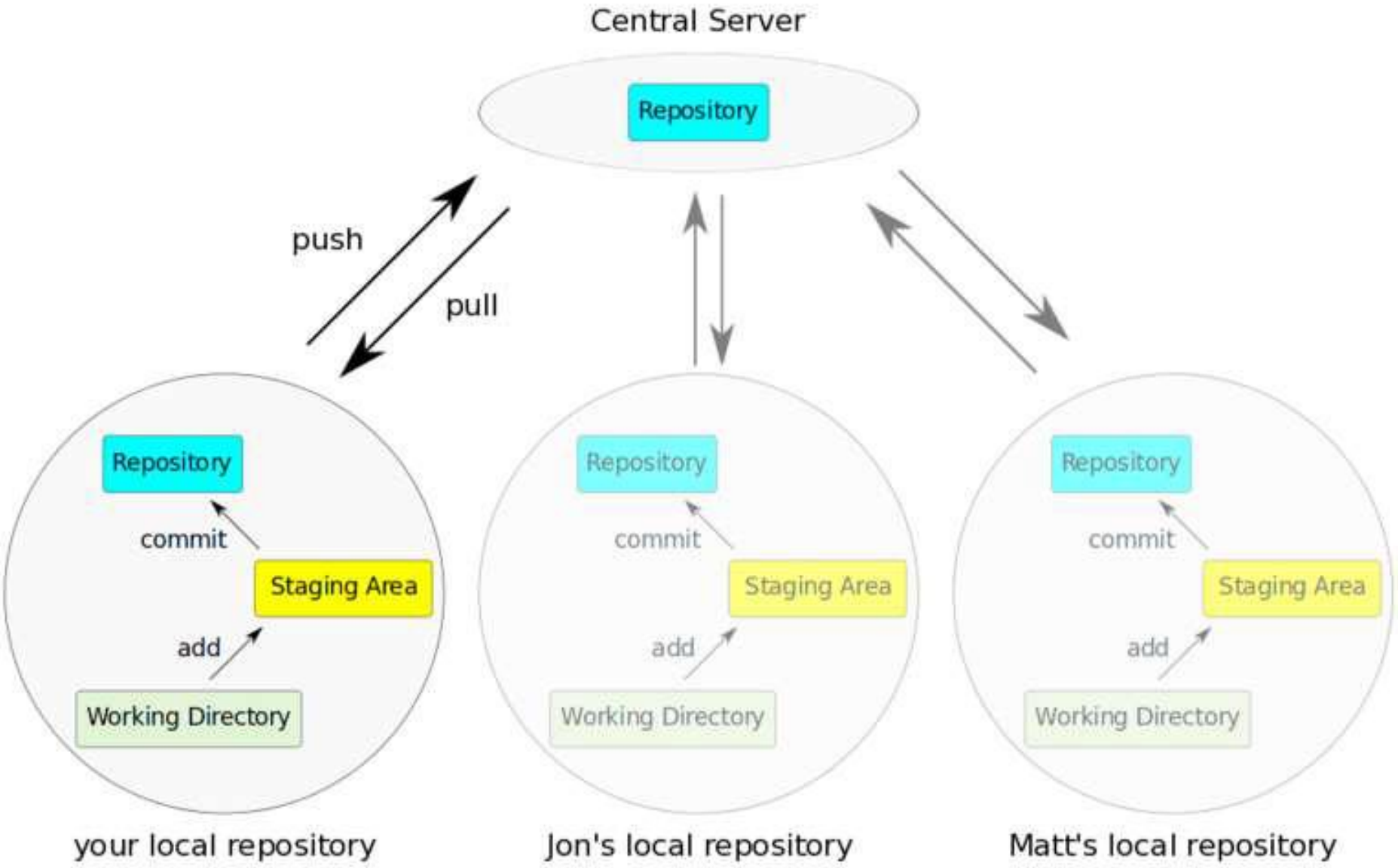
GitHub

- **a platform to host git code repositories**
- http://github.com
- launched in 2008
- most popular Git host
- allows users to collaborate on projects from anywhere
- GitHub makes git social!
- Free to start

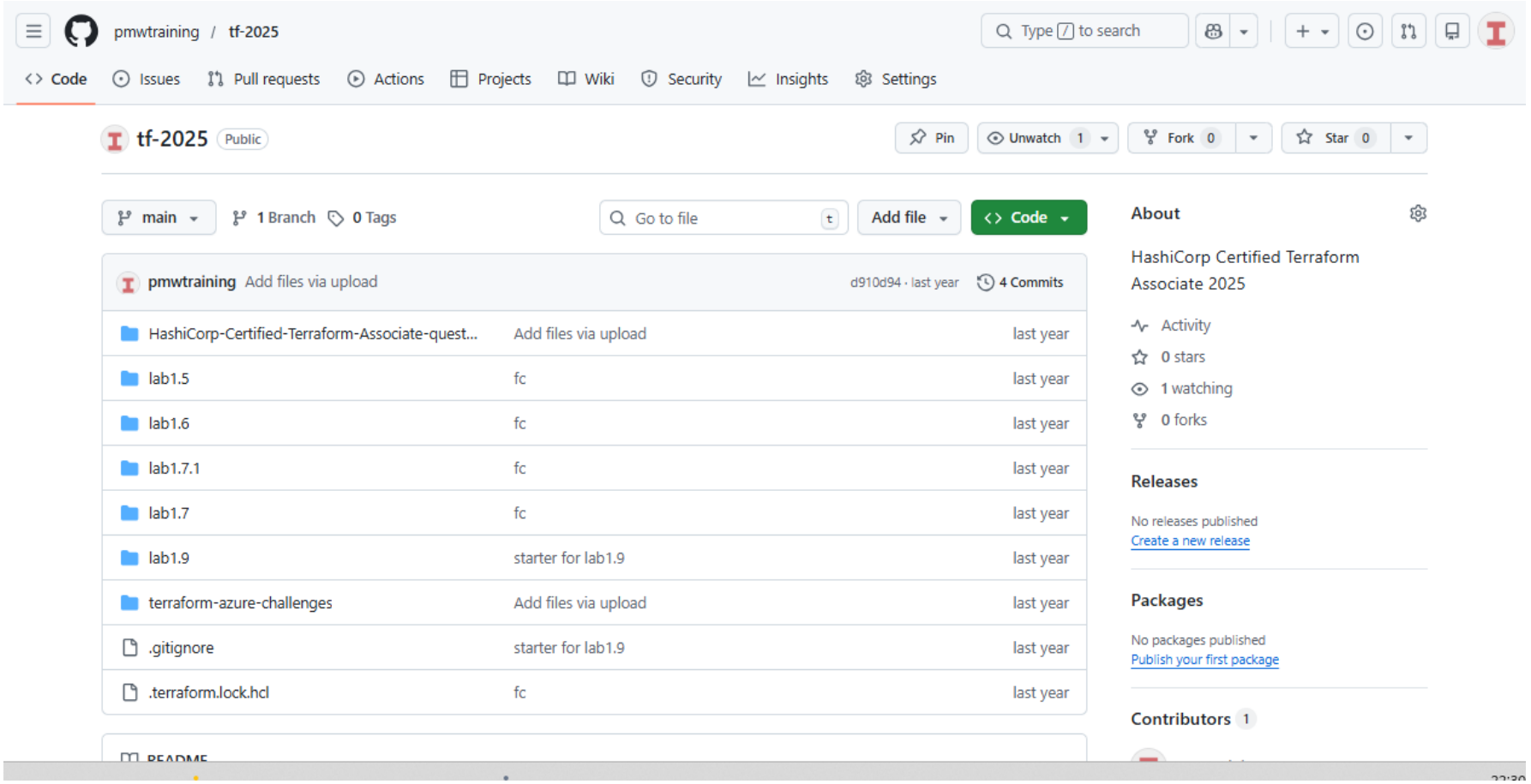

GitHub remote server
Forked branch
fork
master
pull request
someone else's master
pull/ fetch
push
origin/master "branch"
push
merge
merge
repo
origin/master references remote server branch and tries to stay in sync
Local
checkout
commit
Stage
add
Working directory

Important to remember

Sometimes developers *choose* to place repo on GitHub as a centralized place where everyone commits changes, but it **doesn't have to be on GitHub**



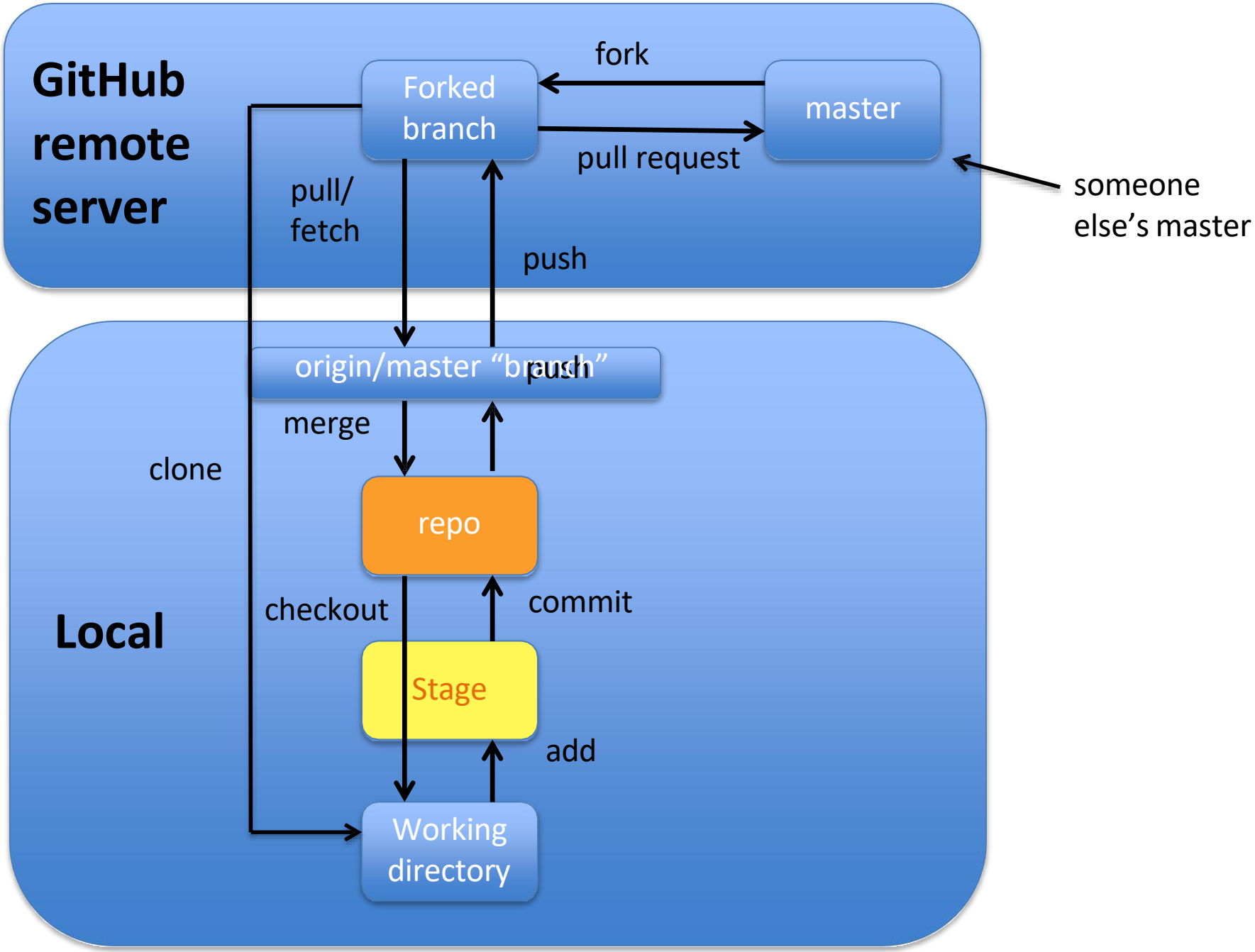
Source: http://bit.ly/1rvzjp9


pmwtraining / tf-2025
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
tf-2025
Public
Pin
Unwatch 1
Fork 0
Star 0
main
1 Branch
0 Tags
Go to file
t
Add file
Code
pmwtraining Add files via upload
d910d94 · last year
4 Commits
HashiCorp-Certified-Terraform-Associate-quest... Add files via upload last year
lab1.5 fc last year
lab1.6 fc last year
lab1.7.1 fc last year
lab1.7 fc last year
lab1.9 starter for lab1.9 last year
terraform-azure-challenges Add files via upload last year
.gitignore starter for lab1.9 last year
.terraform.lock.hcl fc last year
README
About
HashiCorp Certified Terraform Associate 2025
Activity
0 stars
1 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
Contributors 1

Copying (cloning) files from remote repo to local machine

git clone URL <new_dir_name>

```
$ git clone https://github.com/pmwtraining/tf-2025.git
Cloning into 'tf-2025'...
remote: Enumerating objects: 147, done.
remote: Counting objects: 100% (147/147), done.
remote: Compressing objects: 100% (132/132), done.
remote: Total 147 (delta 18), reused 30 (delta 11), pack-reused 0 (from 0)
Receiving objects: 100% (147/147), 1.13 MiB | 3.56 MiB/s, done.
Resolving deltas: 100% (18/18), done.

peter@Ratty MINGW64 ~/gitdemo (master)
$ cd tf-2025/

peter@Ratty MINGW64 ~/gitdemo/tf-2025 (main)
$ ls -la
total 33
drwxr-xr-x 1 peter 197611    0 Mar 29 22:31 ./
drwxr-xr-x 1 peter 197611    0 Mar 29 22:31 ../
drwxr-xr-x 1 peter 197611    0 Mar 29 22:31 .git/
-rw-r--r-- 1 peter 197611   80 Mar 29 22:31 .gitignore
-rw-r--r-- 1 peter 197611 1208 Mar 29 22:31 .terraform.lock.hcl
drwxr-xr-x 1 peter 197611    0 Mar 29 22:31 HashiCorp-Certified-Terraform-Associate-questions/
drwxr-xr-x 1 peter 197611    0 Mar 29 22:31 lab1.5/
drwxr-xr-x 1 peter 197611    0 Mar 29 22:31 lab1.6/
drwxr-xr-x 1 peter 197611    0 Mar 29 22:31 lab1.7/
drwxr-xr-x 1 peter 197611    0 Mar 29 22:31 lab1.7.1/
drwxr-xr-x 1 peter 197611    0 Mar 29 22:31 lab1.9/
drwxr-xr-x 1 peter 197611    0 Mar 29 22:31 terraform-azure-challenges/

peter@Ratty MINGW64 ~/gitdemo/tf-2025 (main)
```


GitHub remote server
Forked branch
fork
master
pull request
someone else's master
pull/ fetch
push
origin/master "branch"
push
merge
clone
repo
checkout
commit
Local
Stage
add
Working directory

How do I link my local repo to a remote repo?

git remote add <alias> <URL>

Note: This just establishes a connection…no files are copied/moved

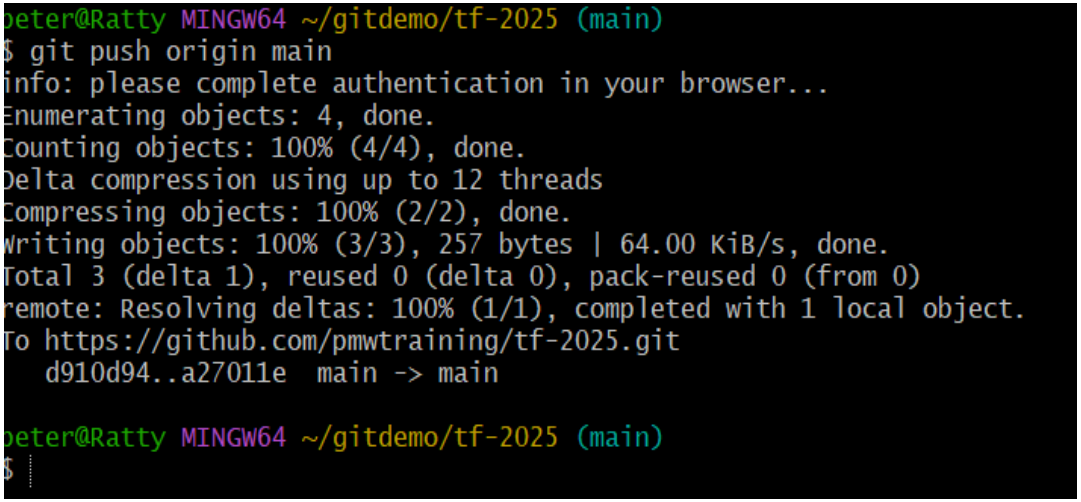
Note: Yes! You may have more than one remote linked to your local directory!

Which remotes am I linked to?

git remote

Pushing to a remote repo

git push local_branch_alias branch_name



Fetching from a remote repo

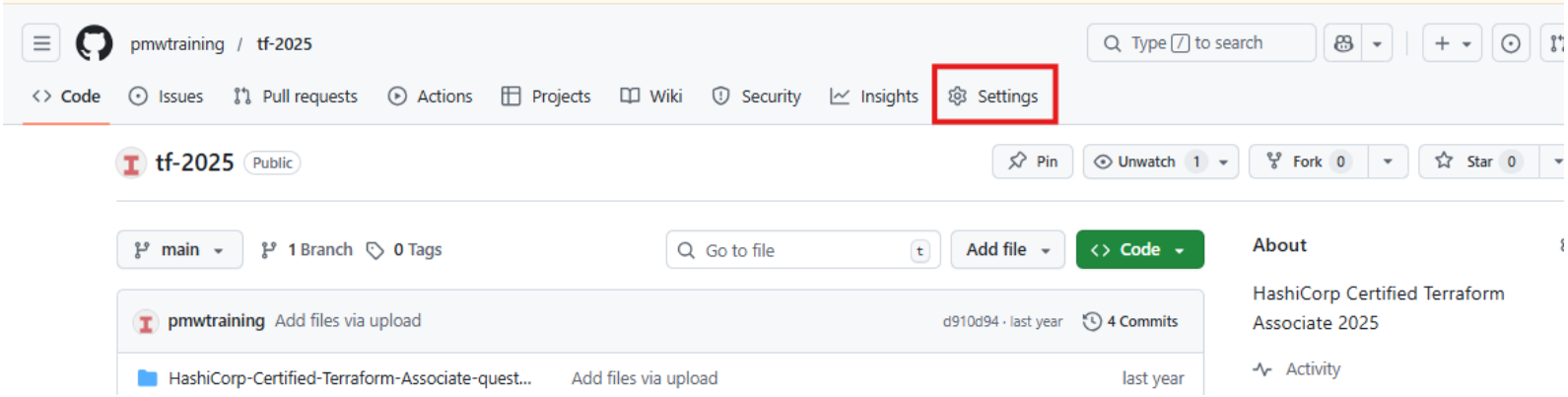
git fetch remote_repo_name

Fetch in no way changes a your working dir or any commits that you've made.

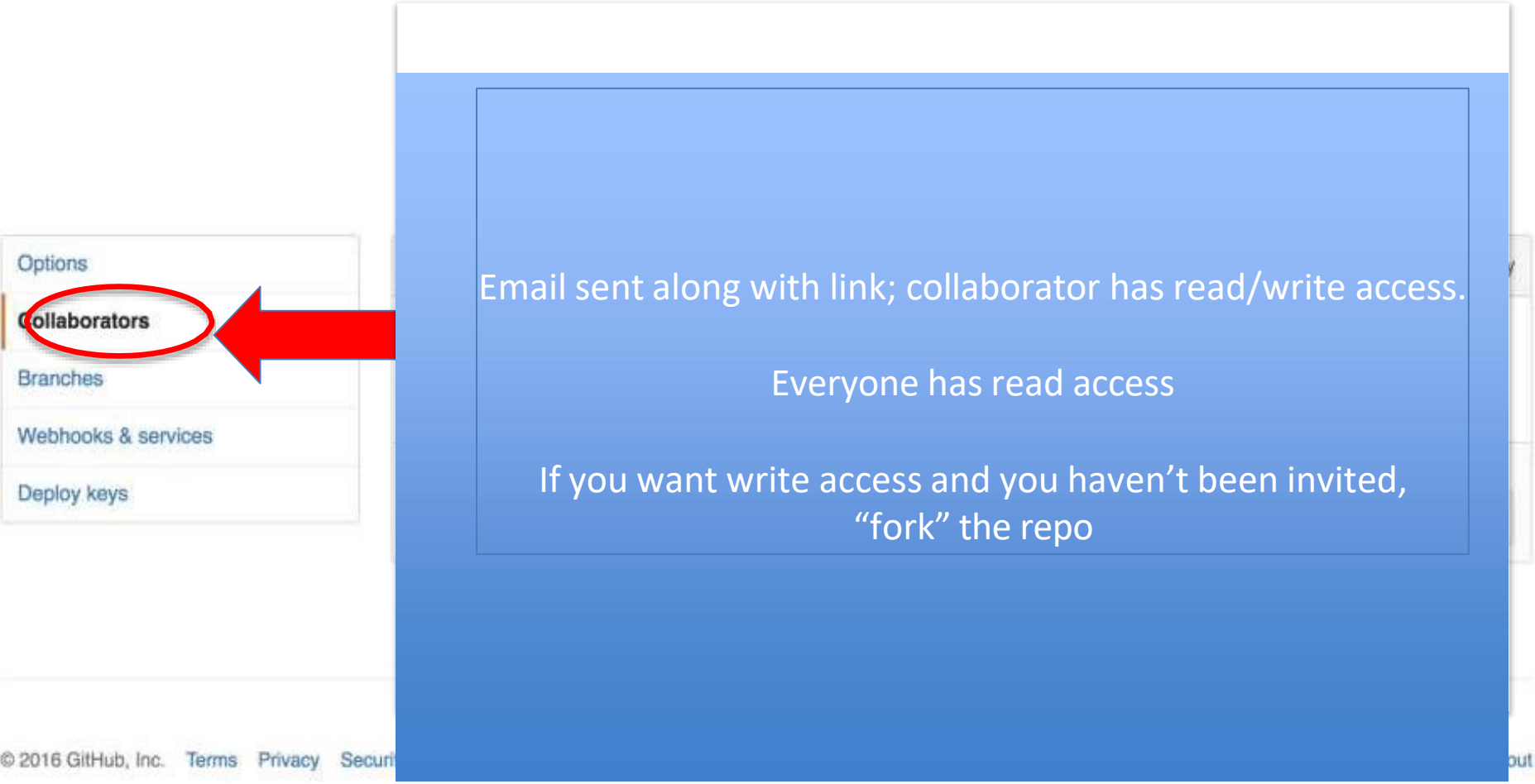
- Fetch before you work
- Fetch before you push
- Fetch often

git merge must be done to merge fetched changes into local branch

Collaborating with Git



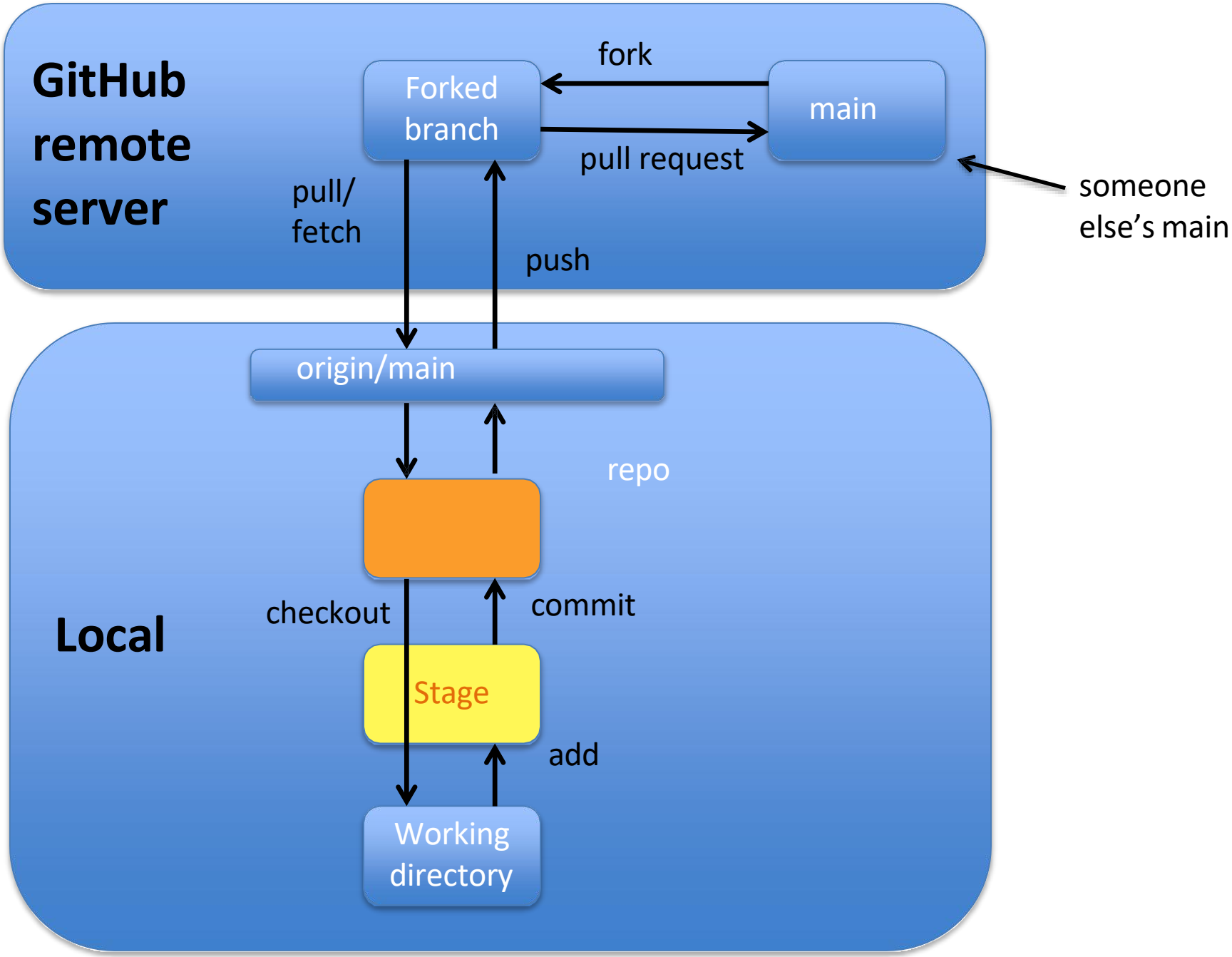
Collaborating with Git



Email sent along with link; collaborator has read/write access.

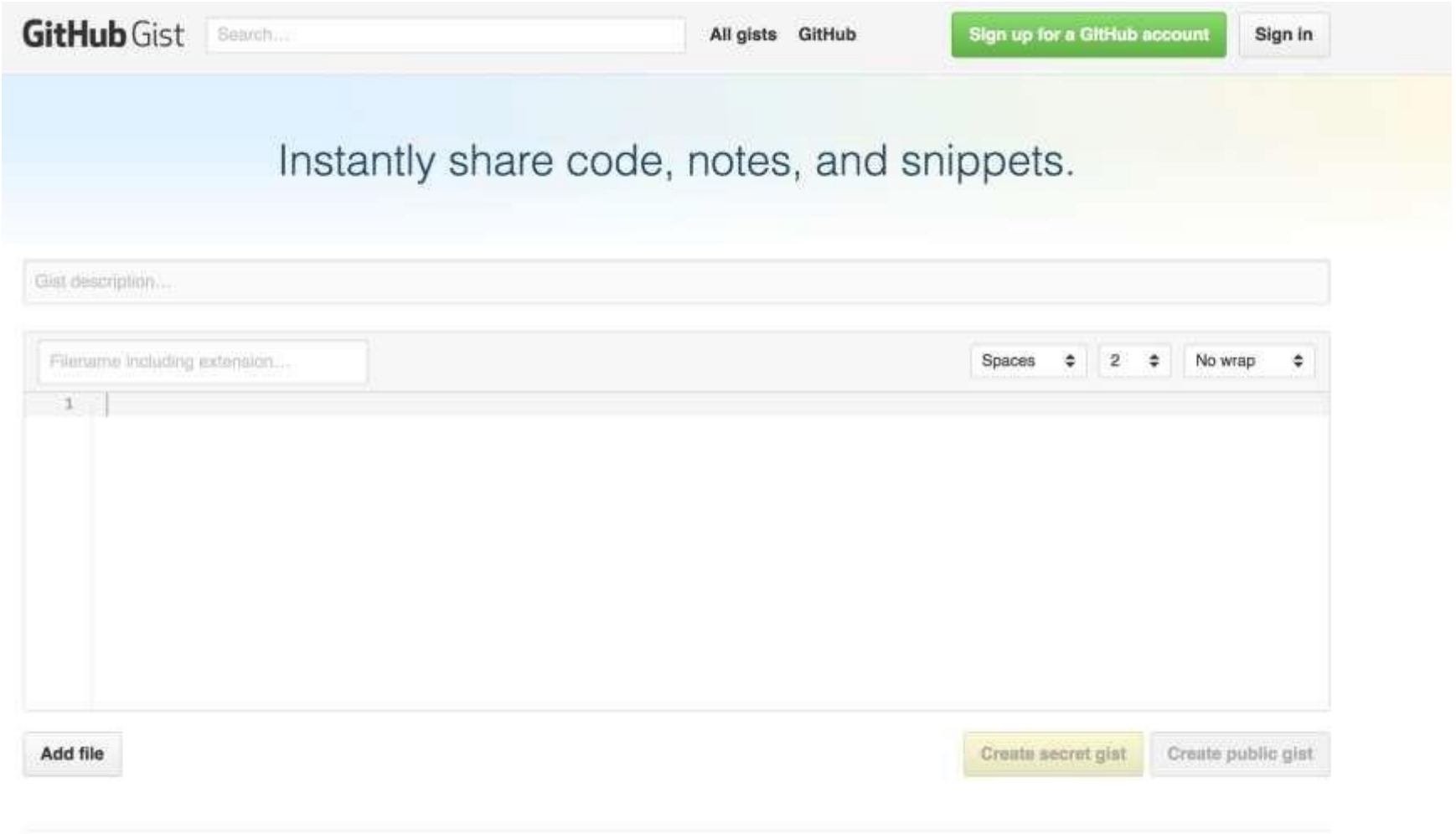
Everyone has read access

If you want write access and you haven't been invited, "fork" the repo


GitHub
remote
server
Forked
branch
fork
main
pull request
someone
else's main
pull/
fetch
push
origin/main
repo
Local
checkout
commit
Stage
add
Working
directory

GitHub Gist

https://gist.github.com/



Good resources

- Git from Git: https://git-scm.com/book/en/v2
- A guided tour that walks through the fundamentals of Git: https://githowto.com
- Linus Torvalds on Git: https://www.youtube.com/watch?v=idLyobOhtO4
- Git tutorial from Atlassian: https://www.atlassian.com/git/tutorials/
- A number of easy-to-understand guides by the GitHub folks https://guides.github.com

git commit -a

- Allows one to add to staging index and commit *at the same time*
- Grabs everything in working direcotry
- Files not tracked or being deleted are not included

git log --oneline

- gets first line and checksum of all commits in current branch

```
$ git log --oneline
a27011e (HEAD -> main, origin/main, origin/HEAD) test
d910d94 Add files via upload
60e76d5 Add files via upload
0d69da7 starter for lab1.9
a1e037d fc
```

git diff **60e76d5**

When using checksum of older commit, will show you all changes compared to those in your working directory

Renaming and deleting branches

git branch –m/--move old_name new_name

git branch –d branch_name

Note: Must not be in branch_name

Note: Must not have commits in branch_name unmerged in branch from which you are deleting

git branch –D branch_name

Note: If you are *really* sure that you want to delete branch with commits

Tagging

- Git has the ability to tag specific points in history as being important, such as releases versions (v.1.0, 2.0, …)

git tag

```
peter@Ratty MINGW64 ~/gitdemo (master)
$ git tag
v1.0
v2.o
```

Tagging

Two types of tags:

lightweight – a pointer to a specific comment – basically a SHA stored in a file

git tag tag_name

annotated – a full object stored in the Git database – SHA, tagger name, email, date, message and can be signed and verified with GNU Privacy Guard (GPG)

git tag –a tag_name –m “message”

How do I see tags?

git show tag_name

```
$ git show v1.4-lw
commit ca82a6dff817ec66f44342007202690a93763949
Author: Scott Chacon <schacon@gee-mail.com>
Date:   Mon Mar 17 21:52:11 2008 -0700

    changed the version number
```

Lightweight tag

```
$ git show v1.4
tag v1.4
Tagger: Ben Straub <ben@straub.cc>
Date:   Sat May 3 20:19:12 2014 -0700

my version 1.4

commit ca82a6dff817ec66f44342007202690a93763949
Author: Scott Chacon <schacon@gee-mail.com>
Date:   Mon Mar 17 21:52:11 2008 -0700

    changed the version number
```

Annotated tag